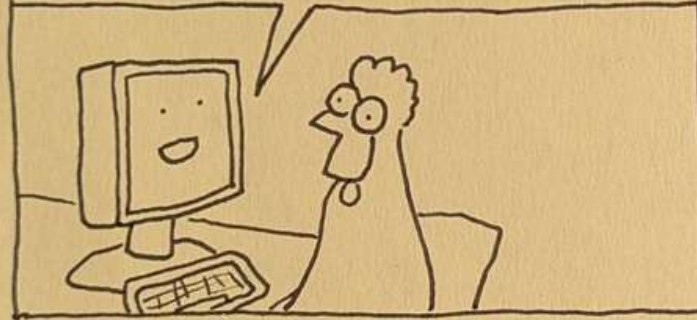
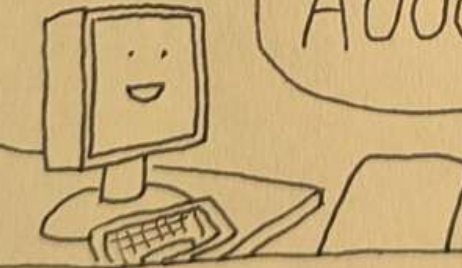


ABSOLUTELY! YOU'VE GOT THIS!

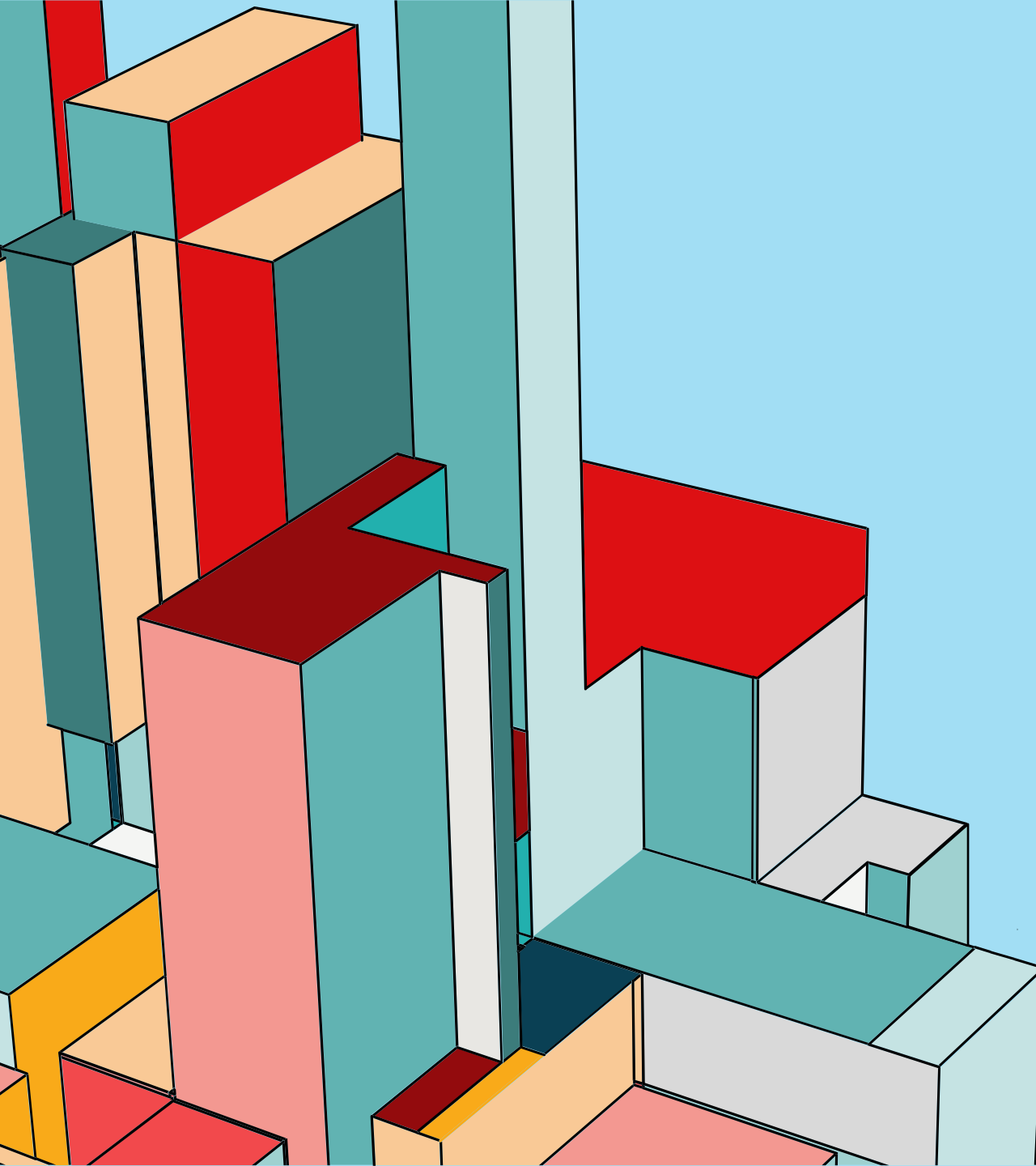


AUUGH!



DON'T ASK A.I. IF YOU SHOULD
TAKE UP CHAINSAW JUGGLING.

© 2026 BY DOUG SAVAGE

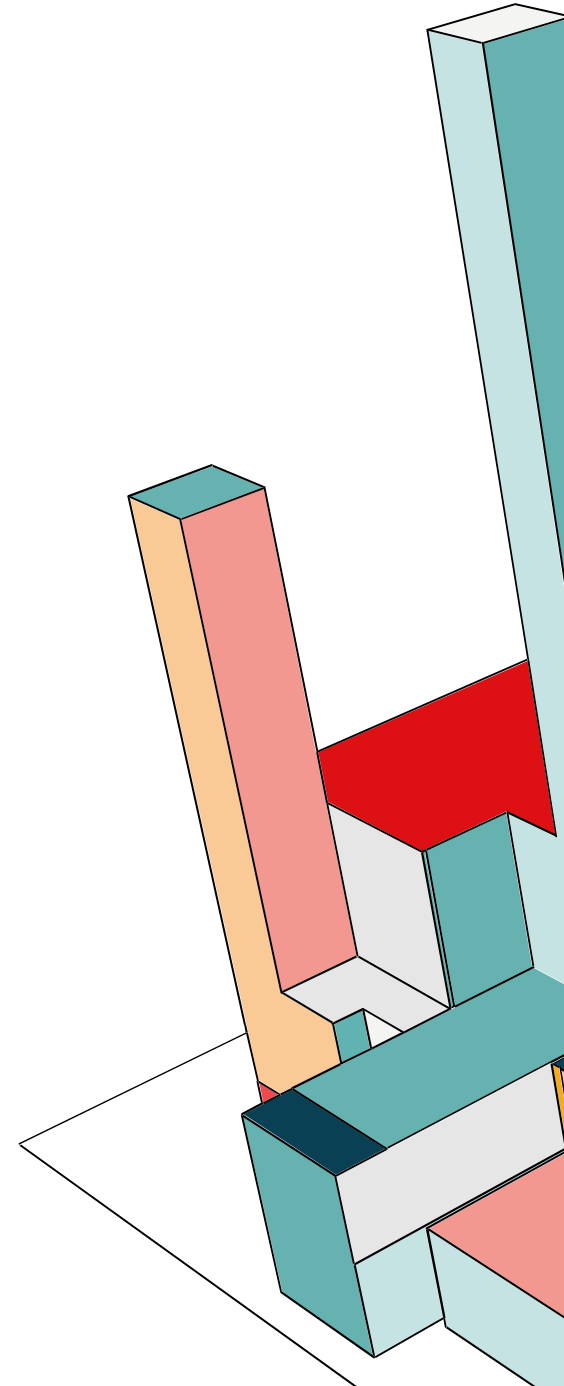


LECTURE 6 **DIVIDE-AND- CONQUER ALGORITHMS**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

ANNOUNCEMENTS

- Exam 1 is on Wed. 2/11, 8-10am.
 - Open paper notes (your own + slides/HW/etc.)
 - More details in future classes.
 - Please reach out with accommodations ASAP.
- No office hours for Dr. DiTursi tomorrow.
 - Email responses may also be sketchy this week.
- Reminder: If you did not finish the lab activities yesterday, please get checked off at any office hours by Tuesday.



TODAY'S TASKS

- Wrap up decrease-and-conquer algorithms
 - Josephus Problem
 - Selection (aka order statistic)
- Divide-and-conquer algorithms
 - Mergesort
 - Master Theorem

THE JOSEPHUS PROBLEM

You have people numbered from 1 to n arranged in a circle. Starting from 1, you alternate between telling people "stay" or "go" – the ones told to go leave the circle. (So 1 stays, 2 goes, 3 stays, etc.) When you complete the circle, you keep going, continuing the same pattern.

Who will be the last person in the circle?

A CONCRETE EXAMPLE

Let $n = 25$. Who is the last person?
Figure it out now.

Enter your answer in the
Submittity poll.



THE JOSEPHUS PROBLEM

As with our previous problems, we can treat this inductively. As with Russian peasant multiplication, we'll need two separate recurrence relations, depending on whether n is odd or even.

When n is even, we'll complete the circle with a problem that is exactly half the size. The survivors will all be in odd positions around the circle; their new positions will be half of their previous ones, rounded up, which we can rewrite:

$$J(k) = J(2k)/2 + 1$$

$$J(2k) = 2 \cdot J(k) - 1$$

THE JOSEPHUS PROBLEM

As with our previous problems, we can treat this inductively. As with Russian peasant multiplication, we'll need two separate recurrence relations, depending on whether n is odd or even.

When n is odd (i.e. $n = 2k + 1$), we'll complete the circle and then tell position 1 to go, leaving us with a problem of size k . Now the new positions are half of the old ones, rounded down. Rewriting gives us:

$$J(k) = J(2k + 1)/2 - 1$$

$$J(2k + 1) = 2 \cdot J(k) + 1$$

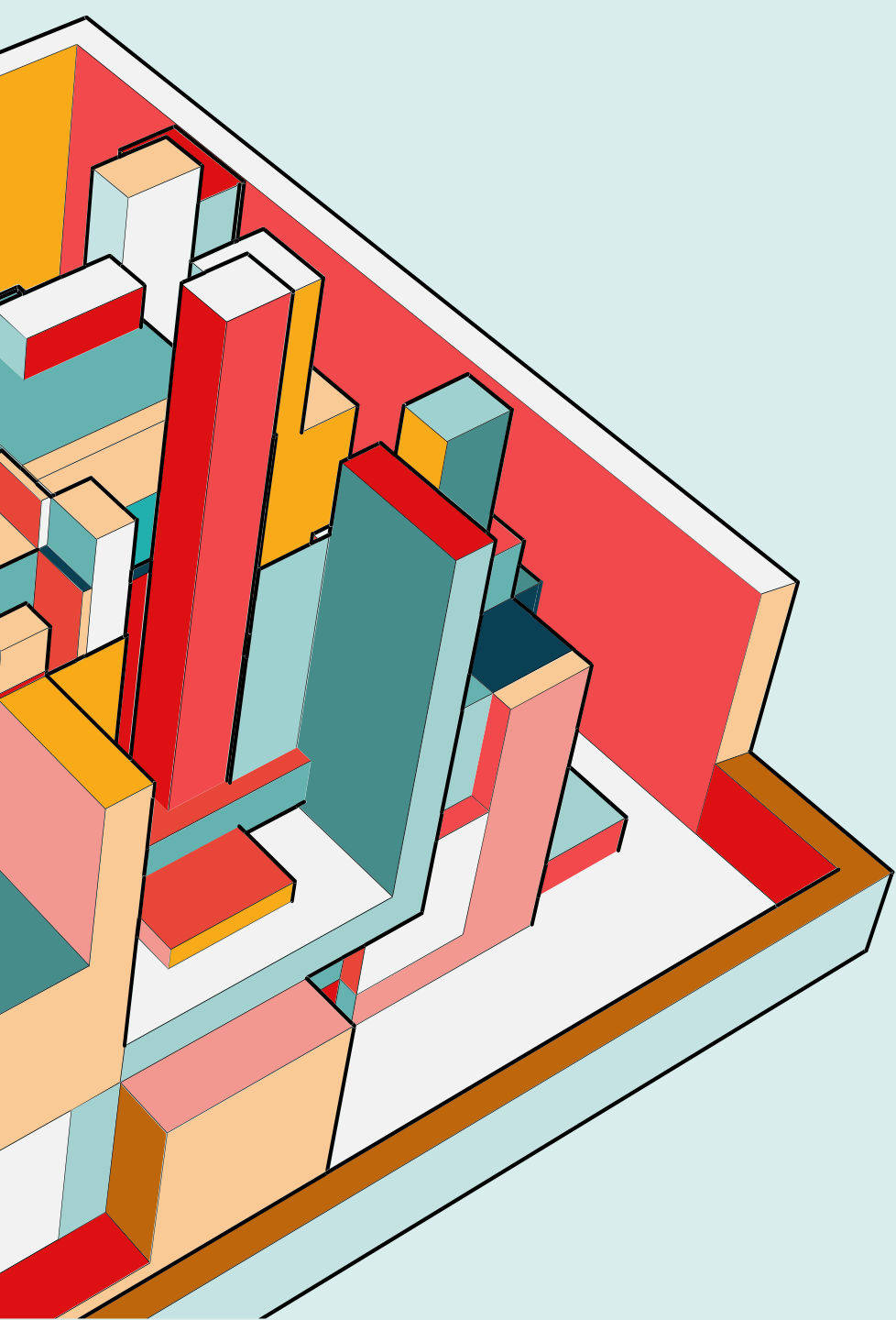
THE JOSEPHUS PROBLEM

$$J(2k) = 2 \cdot J(k) + 1$$

$$J(2k + 1) = 2 \cdot J(k) + 1$$

There is a closed form for these two connected recurrence relations. But simple substitutions won't get us there.

- The answer turns out to be $2r + 1$, where r is the difference between k and the next smallest power of two. ($r = 0$ when k is a power of two.)
- Deriving this result is beyond the scope of this course.
- A lot of work has been done on this problem and variants - a good starting point is in *Concrete Mathematics* by Graham, Knuth, and Patashnik.



SELECTION / ORDER STATISTICS

DEFINING THE PROBLEM

You have a list of n unsorted integers. Find the k -th largest integer in the list. (This is known as the k -th *order statistic*.)

When $k = 1$ or $k = n$, this is easy and takes $\Theta(n)$ time. What about other cases?

To make things concrete for now, we will use $k = \lceil n/2 \rceil$; i.e. we want to find the *median* of the list.

How can we do this efficiently?

Try it now: Come up with an algorithm to find the median.

AN OBVIOUS APPROACH

One possibility is that we can sort the list. The algorithms we have done in this course so far require $O(n^2)$ time; the best* sorting algorithms require $O(n \log n)$ time.

If (and that's a big if) we need to ask this question many times in a row about different k , then this is the best approach.

What if we only need one value (the median)?

It would be a bit weird if we can find the smallest or largest values in linear time, but need loglinear time for the median.

THE CRITICAL IDEA: PARTITIONING

Instead of fully sorting the list (which would be overkill), what if we do a half-assed job of it?

In particular, what we're going to do is pick one element, called the *pivot*, and arrange the list so that every item in the list less than the pivot is on the left, and every item greater than the pivot is on the right.



This process is known as *partitioning* the list.

LOMUTO PARTITIONING

There are two main approaches to partitioning. We'll start with *Lomuto partitioning*.

This is essentially a step-by-one inductive approach. At all times, the list has four sections: the pivot itself, the things less than the pivot, the things greater than the pivot, and the things we haven't looked at yet.

As we examine elements, we either leave them alone (if they're $\geq p$) or we move them to the low end (if $\leq p$).

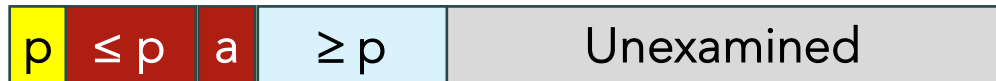
When we're all done, we just swap the pivot with the last low-side element.

LOMUTO PARTITIONING ILLUSTRATION

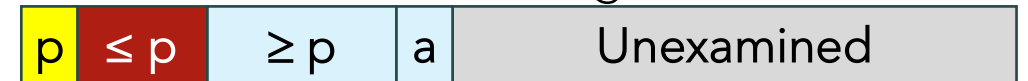
a is the next unexamined element



$a < p$ - swap with
first "big" element



$a \geq p$ - leave in place; a is
now the last "big" element



When we're all done...



Just swap p with the last "small" element (s)

LOMUTO PARTITION PSUEUDOCODE

l and r are the leftmost and rightmost indices

we're interested in; initially 0 and n-1

```
def Lomuto(A,l,r):
```

```
    p = A[l]
```

```
    s = l
```

```
    for i = (l+1) to r:
```

```
        if A[i] < p:
```

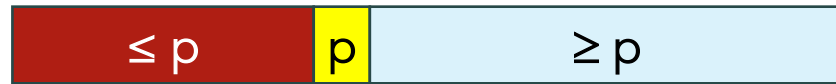
```
            s = s + 1
```

```
            swap(A,s,i) # A[i] becomes last small element
```

```
    swap(A,l,s) # put the pivot in place
```

```
    return s # index of the pivot
```

USING PARTITIONING TO FIND MEDIAN



How does partitioning help us find the order statistic?

We check the index i of the pivot when we're done.

- If $k = i + 1$, we're done! Return p .
 - *Note*: $i + 1$ because array indices start at 0; e.g., 5th smallest is $A[4]$
- If $k < i + 1$, we should look in the small half of the list. Our target will be the k -th smallest element there.
- If $k > i + 1$, we should look in the big half of the list. Our target will be the $(k - (i + 1))$ -th smallest element there.

QUICKSELECT PSEUDOCODE

```
def quickselect(A,l,r,k):  
    s = Lomuto(A,l,r)  # or other partition algorithm  
    if k == s+1 return A[s]  
    if k < s+1:  
        return quickselect(A,l,r-1,k)  
    else:  
        return quickselect(A,l+1,r,k-(s+1))
```

QUICK PRACTICE

Show the operation of Quickselect to find the median of this 9-element list: 4, 1, 10, 8, 7, 12, 9, 2, 15

First we observe $k = 5$. (The median is the fifth element.)

4	1	10	8	7	12	9	2	15
---	---	----	---	---	----	---	---	----

4	1	10	8	7	12	9	2	15
---	---	----	---	---	----	---	---	----

4	1	10	8	7	12	9	2	15
---	---	----	---	---	----	---	---	----

4	1	10	8	7	12	9	2	15
---	---	----	---	---	----	---	---	----

4	1	10	8	7	12	9	2	15
---	---	----	---	---	----	---	---	----

4	1	10	8	7	12	9	2	15
---	---	----	---	---	----	---	---	----

4	1	10	8	7	12	9	2	15
---	---	----	---	---	----	---	---	----

4	1	2	8	7	12	9	10	15
---	---	---	---	---	----	---	----	----

4	1	2	8	7	12	9	10	15
---	---	---	---	---	----	---	----	----

2	1	4	8	7	12	9	10	15
---	---	---	---	---	----	---	----	----

QUICK PRACTICE

2	1	4	8	7	12	9	10	15
---	---	---	---	---	----	---	----	----

The return value is 2, and $5 > 2+1$, so we must look in the larger half. Our new k value is $5 - (2 + 1) = 2$.

2	1	4	8	7	12	9	10	15
---	---	---	---	---	----	---	----	----

2	1	4	8	7	12	9	10	15
---	---	---	---	---	----	---	----	----

2	1	4	8	7	12	9	10	15
---	---	---	---	---	----	---	----	----

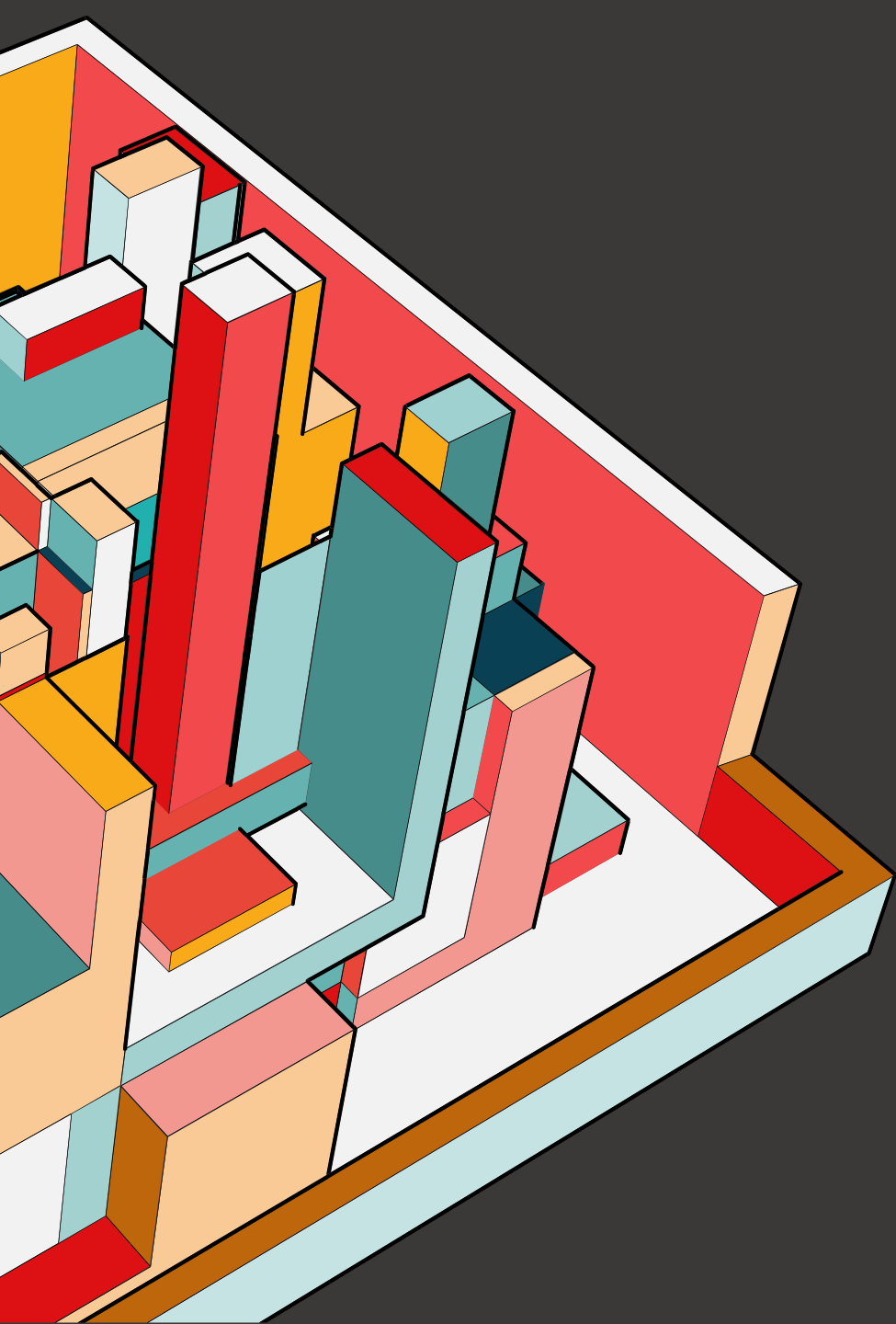
2	1	4	8	7	12	9	10	15
---	---	---	---	---	----	---	----	----

2	1	4	8	7	12	9	10	15
---	---	---	---	---	----	---	----	----

2	1	4	8	7	12	9	10	15
---	---	---	---	---	----	---	----	----

2	1	4	7	8	12	9	10	15
---	---	---	---	---	----	---	----	----

This time the return value is 1, and $2 = (1+1)$, so we have found our median: 8.



QUICK BREAK

QUICKSELECT: RUNTIME ANALYSIS

How “quick” is Quickselect?

In the best case, our initial pivot is the target element. In this case, we only have to perform $n - 1$ comparisons. (Notice this is the same as finding the smallest/largest element.)

Consider this scenario: The list is already sorted, but you don’t know that. You run Quickselect for the median. How does it do?

The first round of partitioning does $n - 1$ comparisons and moves... nothing at all! The return value is 0, so reduce the size by one spot.

... and then the same thing happens again, and again, and again, until you finally hit the median. $\frac{n}{2}$ partitions with $\geq \frac{n}{2}$ comparisons each. $O(n^2)$.

THE IMPORTANCE OF A GOOD PIVOT

The performance of Quickselect is critically dependent on the partitions dividing up the list into roughly equal parts.

In general, this requires being at least slightly clever about selecting our pivot element. Middle-of-three (look at three elements, use the center for your pivot) is a simple approach that usually avoids really bad cases.

There are super-clever methods that can mathematically guarantee a good split, but these are so complex that they are almost never worth actually using in practice.

If partitioning produces roughly equal halves, what can we conclude about an average-case runtime?

QUICKSELECT: “AVERAGE” CASE

Let us presume for the moment that each partition divides the list perfectly in half, but we’re looking for an arbitrary k (not necessarily the median).

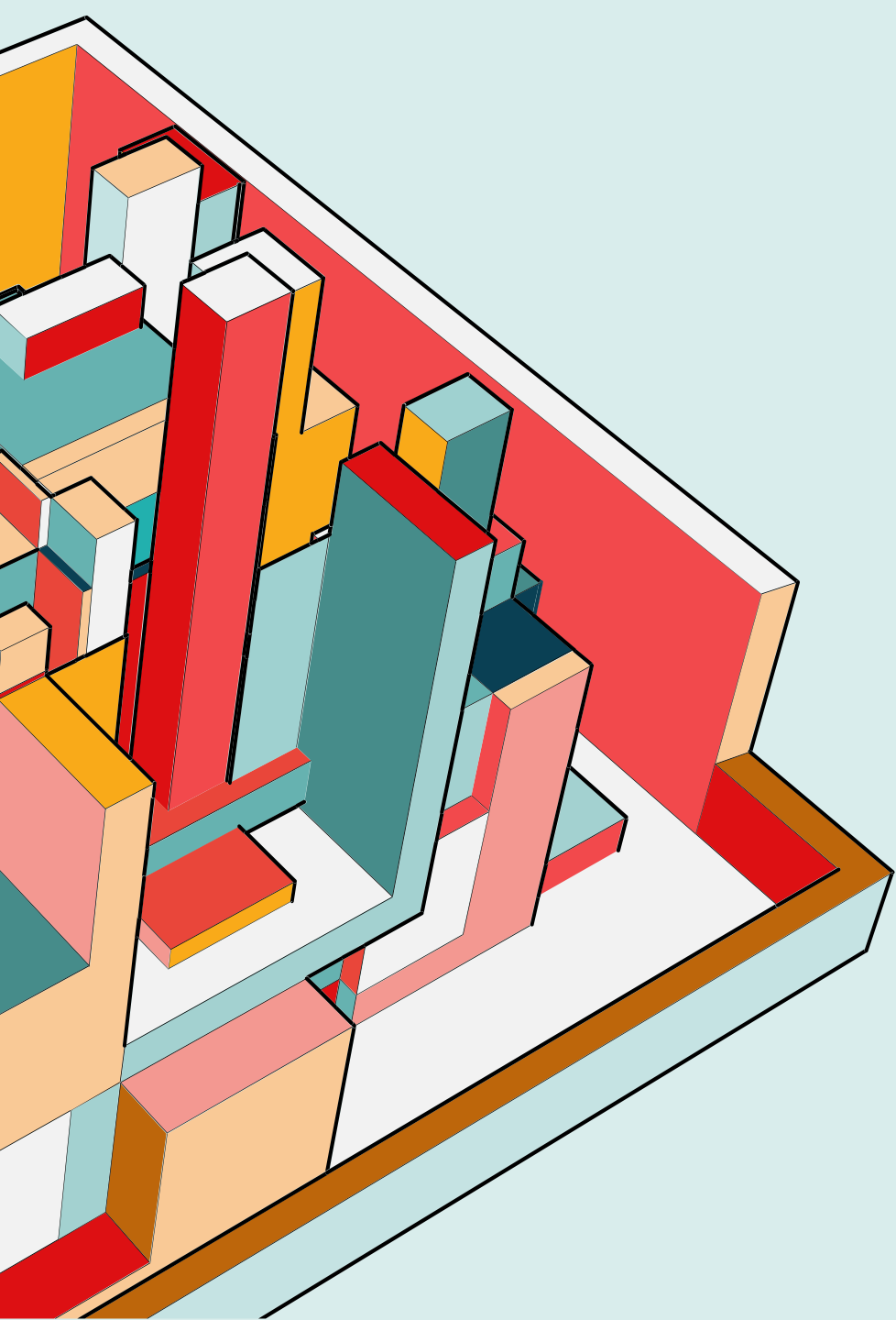
The first partition will need $\sim n$ comparisons, the next one will need $n/2$, then $n/4$, potentially all the way down to 1.

What is $\sum 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$?

- Answer: 2

When partition works ideally, Quickselect is $\Theta(n)$ for any k .

Turns out we don’t need ideal – so long as we peel off at least a constant fraction each time, Quickselect remains linear.



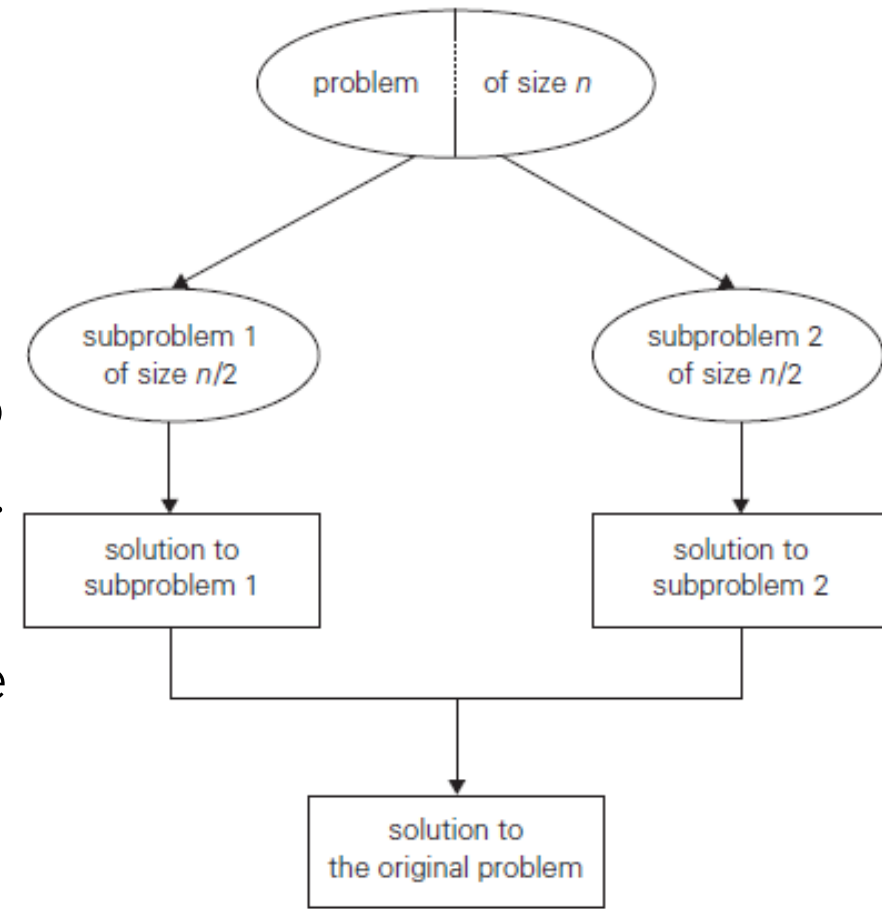
DIVIDE-AND- CONQUER ALGORITHMS

WHAT IS “DIVIDE-AND-CONQUER”?

Honestly, it's a lot like the inductive algorithms we've seen so far. As before, we're going to solve a problem by solving smaller versions of the same problem.

The key difference is that we generally need to solve more than one subproblem at each step.

In contrast, binary search splits the search space in half, but then we throw out the half we don't care about – we only need one recursive call at each step.

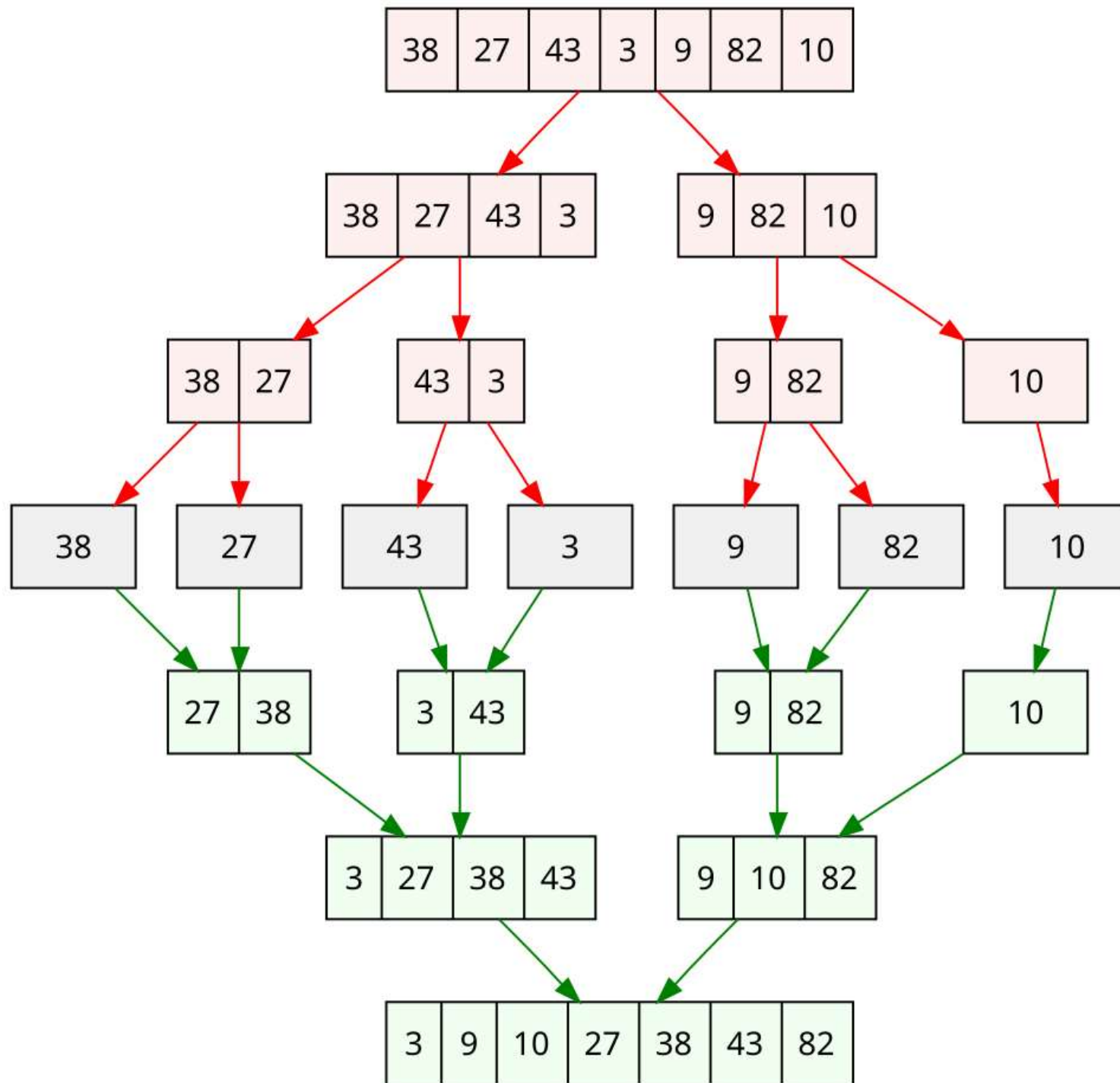


CANONICAL EXAMPLE: MERGESORT

As a reminder, the mergesort algorithm works as follows:

- Split the list in half. (Unless the list is one element, then just return.)
- Recursively mergesort the two halves.
- *Merge* the two sorted halves into a single sorted list.

The merge process scans down both lists from left to right, selecting the smaller element at each stage, copying it into the merged copy, and advancing only the pointer for the list containing the selected element.



Mergesort is the archetype (the Trope Maker, if you will) of the divide-and-conquer approach. We take a problem of size n , split it into two problems of about size $n/2$, and then solve those smaller problems recursively. At the end, we combine the two small solutions into a full solution.

MERGESORT PSEUDOCODE

```
def mergesort(A,l,r):  # l,r are again left, right
    if (r-l) <= 0 return  # list has <= 1 elements
    m = (l + r) // 2  # midpoint
    mergesort(A,l,m)
    mergesort(A,m+1,r)
    merge(A,l,m,r)
```

MERGE PSEUDOCODE

```
def merge(A, l, m, r):  
    Create empty array C with (r-l+1) spots  
    i = l; j = m+1; k = 0  
    while (i <= m) and (j <= r):  
        if A[i] < A[j]:  
            C[k] = A[i] and advance i by 1  
        else:  
            C[k] = A[j] and advance j by 1  
        Advance k by 1  
    Copy whichever side isn't done onto the end of C  
    Copy C back into A[l..r]
```

MERGESORT RUNTIME

Since mergesort is a recursive algorithm, our standard approach is to write a recurrence relation describing the amount of work done by mergesort. [Write one now.](#)

```
def mergesort(A,l,r):    # l,r are again left, right
    if (r-l) <= 0 return # list has <= 1 elements
    m = (l + r) // 2    # midpoint
    mergesort(A,l,m)
    mergesort(A,m+1,r)
    merge(A,l,m,r)
```

SUBMITTY POLL #2

What is the recurrence relation that describes mergesort?



MERGESORT RUNTIME

```
def mergesort(A,l,r):  # l,r are again left, right
    if (r-l) <= 0 return  # list has <= 1 elements
    m = (l + r) // 2  # midpoint
    mergesort(A,l,m)
    mergesort(A,m+1,r)
    merge(A,l,m,r)
```

Examining the code, we make two recursive calls, each of which operates on a list that is half the size of the original, giving: $T(n) = 2T(n/2) + ???$.

A key question becomes: How much work does merge do?

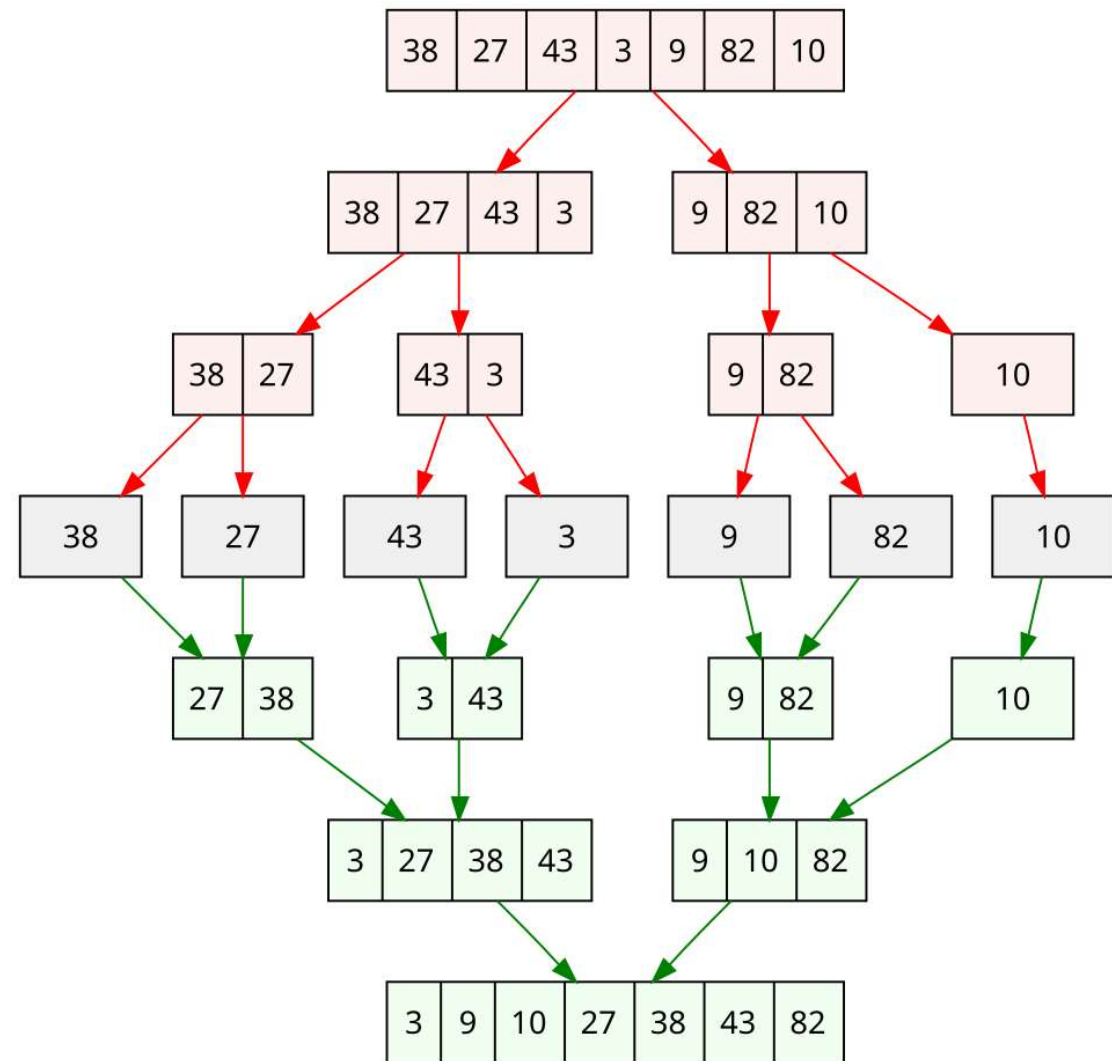
MERGESORT RUNTIME

We can reason that merge does a constant amount of work *per element that it merges*. (One comparison, two copies.)

Thus, merge is $\Theta(n)$, and our overall recurrence is:

$$T(n) = 2T(n/2) + cn$$

Does this have a closed form?
Can we find it?



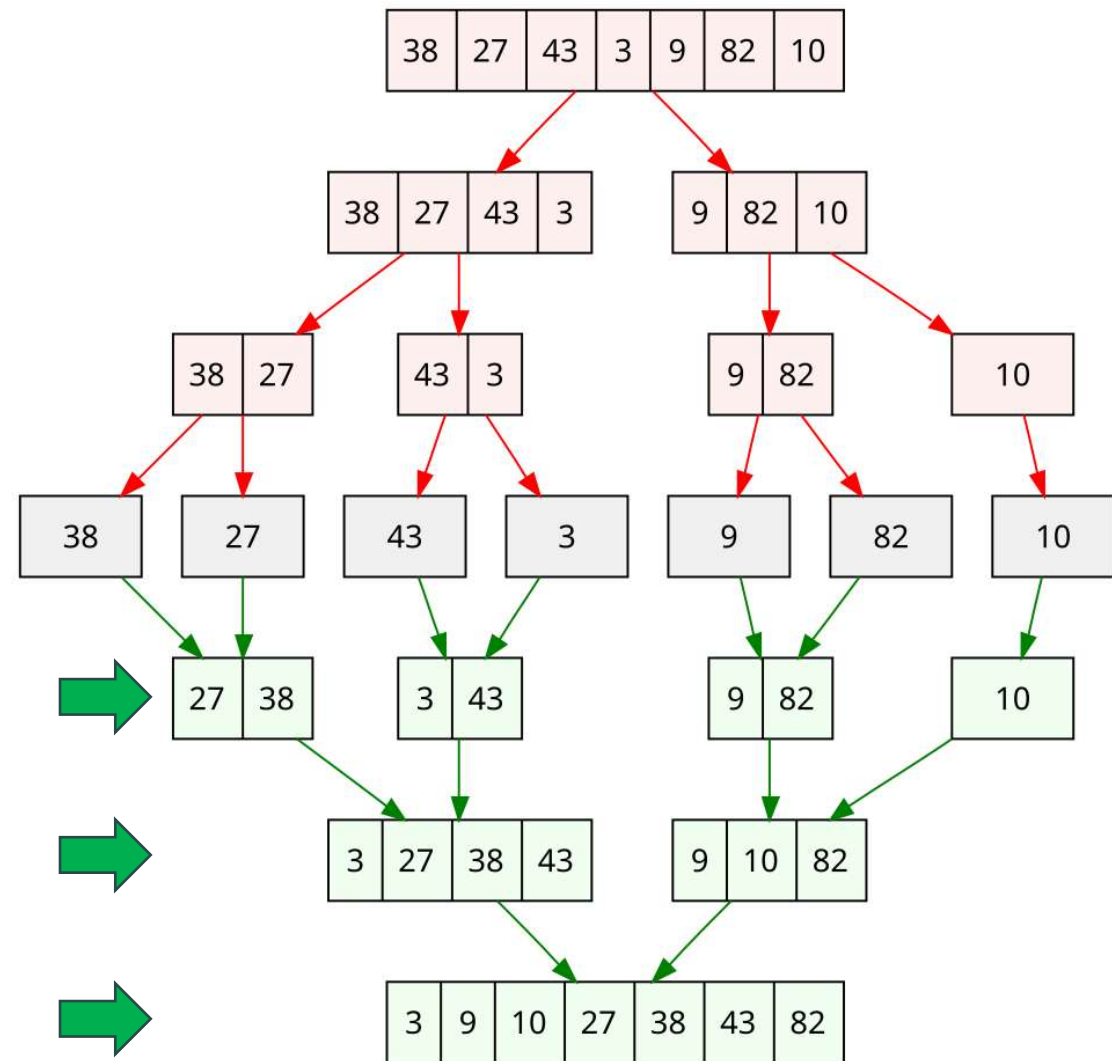
MERGESORT RUNTIME

The key observation is that at each layer of recursive calls, we merge each element of the full list one time.

Thus, at each layer, we do $\Theta(n)$ work. How many layers are there?

Answer: $\lg n$

Thus, the overall mergesort algorithm is $\Theta(n \log n)$.



ANOTHER EXAMPLE

Consider the following algorithm:

```
def Bob(n):  
    do_thing()  
    if n > 1:  
        Bob(n/2)  
        Bob(n/2)
```

How many times (in terms of n) will it call **do_thing()**? Write the recurrence relation, then draw a tree to try to figure out a closed form.

ANOTHER EXAMPLE

```
def Bob(n):  
    do_thing()  
    if n > 1:  
        Bob(n/2)  
        Bob(n/2)
```

The correct recurrence is $F(n) = 1 + 2F(n/2)$.

This is like mergesort, but we only do a constant amount of work at each layer. We could draw out a tree again, but surely there is a more... precise way of getting a closed form.

There is! **THE MASTER THEOREM!**



THE MASTER THEOREM

Despite the rather grandiose title, the Master Theorem isn't that big of a deal. It provides a closed-form solution to most recurrence relations where the recursion decreases by a constant factor.

Given a recurrence relation of the form

$T(n) = aT(n/b) + f(n); T(1) = c$ where $f(n) = \Theta(n^d)$, then:

$$T(n) = \begin{cases} \Theta(n^d) & \text{when } a < b^d \\ \Theta(n^d \log n) & \text{when } a = b^d \\ \Theta(n^{\log_b a}) & \text{when } a > b^d \end{cases}$$

THE MASTER THEOREM

$T(n) = aT(n/b) + f(n); T(1) = c$ where $f(n) = \Theta(n^d)$, then:

$$T(n) = \begin{cases} \Theta(n^d) & \text{when } a < b^d \\ \Theta(n^d \log n) & \text{when } a = b^d \\ \Theta(n^{\log_b a}) & \text{when } a > b^d \end{cases}$$

This can use a bit of interpretation. Basically:

- The bottom case says the number of recursive calls (the width of the recursive tree) dominates the runtime.
- The top case says the work we are doing within each recursive call (i.e. the non-recursive bits) dominates.
- The middle case says these two factors are in some sense balanced, and we must consider both of them.

THE MASTER THEOREM

$T(n) = aT(n/b) + f(n); T(1) = c$ where $f(n) = \Theta(n^d)$, then:

$$T(n) = \begin{cases} \Theta(n^d) & \text{when } a < b^d \\ \Theta(n^d \log n) & \text{when } a = b^d \\ \Theta(n^{\log_b a}) & \text{when } a > b^d \end{cases}$$

Recall that **mergesort** was $F(n) = 2F(n/2) + cn$.

This gives us $a = 2, b = 2, d = 1$, which means we're in the middle case. We end up with $\Theta(n \log n)$, which agrees with the tree-based approach we used earlier.

For Bob, on the other hand, $a = 2, b = 2, d = 0$, putting us in the bottom case. We get $\Theta(n)$.

BEFORE YOU LEAVE

